R1.04 — Systèmes d'exploitation & Fonctionnement : Processus, Filiation et Signaux POSIX

Thème principal : Systèmes d'exploitation et gestion des processus sous Unix/Linux

Synthèse exécutive : Ce cours explore la transition fondamentale entre un programme statique stocké sur disque et un processus dynamique en mémoire RAM sous Unix/Linux. Il détaille le rôle de l'ordonnanceur du noyau, le modèle d'états à trois niveaux (Élu, Prêt, Bloqué), la filiation arborescente des processus via les appels système fork() et exec(), ainsi que la communication inter-processus par le biais des signaux POSIX.

Du Programme Statique au Processus Dynamique : Cycle de Vie et Multitâche

Pour exécuter un code informatique, le système d'exploitation fait basculer le programme (données passives sur disque) vers l'état de processus (entité active en mémoire RAM). L'ordonnanceur du noyau orchestre cette activité via un modèle à trois états fondamentaux. Lorsqu'un processus est à l'état 'Élu', il occupe physiquement un cœur de processeur et ses instructions sont exécutées. Si son quantum de temps CPU expire, le noyau le bascule à l'état 'Prêt' et attribue le CPU à un autre processus. Si le processus déclenche une demande de lecture sur disque ou attend une saisie clavier, il bascule à l'état 'Bloqué' afin de ne pas gaspiller inutilement des cycles CPU. Dès que la ressource demandée devient disponible, le noyau bascule le processus à l'état 'Prêt' (et non directement à l'état 'Élu'), le replaçant dans la file d'attente de l'ordonnanceur. Chaque processus possède un identifiant unique (PID) et référence le PID de son créateur (PPID). La création d'une sous-tâche s'appuie sur l'appel système fork() qui duplique la structure du parent, suivi de l'appel exec() pour charger le nouveau binaire.

- Programme vs Processus : Différence entre fichier exécutable passif et instance active en RAM.
- Multitâche Préemptif : Découpage du temps CPU en quanta par l'ordonnanceur.
- PCB (Process Control Block) : Structure de données interne regroupant les métadonnées (PID, PPID, état, registres).
- États du processus : Élu, Prêt, Bloqué.

Cas concret enseignant : *L'analogie du livre de cuisine : Le programme stocké sur disque dur est comparé à un livre de recettes sagement rangé dans une bibliothèque (passif, statique, ne consommant ni électricité ni ingrédients). Le processus est le cuisinier en train de préparer activement le plat dans sa cuisine (dynamique, consommant de l'espace en RAM sur le plan de travail, du temps processeur CPU pour la cuisson et utilisant des ustensiles correspondant aux descripteurs de fichiers).*

Contrôle et Communication Inter-Processus : Les Signaux POSIX

Les signaux constituent la forme la plus basique et universelle de communication inter-processus (IPC) sous les systèmes POSIX. Lorsqu'un signal est transmis à un processus cible, le noyau interrompt le déroulement séquentiel des instructions pour forcer le traitement du signal. Par défaut, la plupart des signaux entraînent la terminaison du processus. Un développeur peut capturer un signal via un signal handler. Toutefois, pour garantir le contrôle des programmes défaillants, les signaux SIGKILL (9) et SIGSTOP (19) échappent à ce mécanisme et sont exécutés directement par le noyau.

- Signal POSIX : Interruption logicielle asynchrone transmise par le noyau.
- Gestionnaire de signal (Signal Handler) : Fonction personnalisée pour intercepter un signal.
- SIGINT (Signal 2) : Interruption modérée générée par Ctrl + C.
- SIGTERM (Signal 15) : Demande de terminaison propre.
- SIGKILL (Signal 9) : Arrêt inconditionnel géré par le noyau, inaltérable.

Cas concret enseignant : *L'analogie de l'évacuation de l'immeuble : SIGTERM (Signal 15) est le gardien d'immeuble qui frappe poliment à la porte pour vous demander de quitter les lieux (vous disposez du temps nécessaire pour ranger votre bureau et sauvegarder vos travaux). SIGKILL (Signal 9) représente l'intervention des forces de sécurité qui défoncent la porte et vous expulsent sur-le-champ sans vous laisser la possibilité de toucher à quoi que ce soit.*

■ Questions & Interventions des Étudiants en Classe

Question étudiant : Pourquoi la commande 'kill 9 1234' renvoie-t-elle une erreur dans le terminal alors que 'kill -9 1234' fonctionne correctement ?
Réponse du professeur : La commande 'kill' attend une option précédée d'un tiret (comme '-9' pour spécifier le signal SIGTERM ou SIGKILL). Sans le tiret, le shell interprète le premier argument '9' comme un identifiant de signal passé sans formatage correct ou comme un PID erroné selon la syntaxe, d'où l'erreur de syntaxe ou de paramétrage.

Question étudiant : Qu'est-ce qu'un processus dit 'Zombie' et comment le détruit-on avec la commande kill ?
Réponse du professeur : Un processus zombie est un processus qui a terminé son exécution mais dont l'entrée dans la table des processus (le PCB) subsiste car le processus parent n'a pas encore lu son code de retour (via l'appel system wait()). Étant donné qu'un zombie est déjà mort et ne consomme plus de ressources CPU ni mémoire (sauf une entrée dans la table des processus), la commande 'kill' n'a aucun effet sur lui. Pour s'en débarrasser, il faut tuer son processus parent afin que le processus init/systemd adopte le zombie et nettoie sa table.

■ Exercice & Démonstration d'Application

Énoncé : Dans un terminal Bash Linux, un étudiant saisit la séquence de commandes suivante : 1. sleep 600 & ; 2. ps -ef | grep sleep ; 3. kill -15 . Explicitez le rôle du symbole '&', détaillez le comportement produit lors de l'envoi du signal -15 et décrivez le résultat obtenu lors d'un contrôle ultérieur par la commande ps.

Résolution pas à pas :
Étape 1 : Le symbole '&' placé à la fin de 'sleep 600 &' indique au terminal d'exécuter la commande en arrière-plan (background). Le shell affiche l'identifiant de job ainsi que le PID attribué par le noyau (ex: 4521) puis rend immédiatement la main à l'utilisateur. Étape 2 : La commande 'ps -ef | grep sleep' parcourt la liste globale des processus système et filtre le mot-clé 'sleep', affichant le PID (4521) et le PPID. Étape 3 : La commande 'kill -15 4521' transmet le signal SIGTERM (Signal 15) au processus 4521. L'application 'sleep' s'arrête proprement.

Résultat : Le processus 'sleep 600' est détruit de manière propre. Une seconde vérification au moyen de 'ps -ef | grep sleep' confirme la disparition totale du processus de la table des processus actifs.

■ Auto-Évaluation Flash d'Examen

Q : Quel est le nom et le PID du tout premier processus utilisateur démarré par le noyau Linux au lancement de la machine ?

R : systemd (ou init), possédant le PID 1.

Q : Quel signal POSIX est généré et transmis au processus en premier plan lorsque l'utilisateur effectue le raccourci 'Ctrl + C' dans le terminal ?

R : Le signal SIGINT (Signal 2).

Q : Pourquoi le signal SIGKILL (Signal 9) ne peut-il jamais être intercepté ou ignoré par un programme informatique ?

R : Parce qu'il est géré directement par le noyau Linux pour garantir à l'administrateur la possibilité absolue de stopper un processus bloqué, corrompu ou malveillant.